



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 12
по курсу «Языки и методы программирования»
«FileExplorer. Сокеты и Swing GUI»

Студент группы ИУ9-22Б
Булдаков А. С.

Преподаватель
Посевин Д. П.

Москва 2026

Содержание

Цель работы	1
Серверная часть	1
Класс FileExplorer	1
Функция parse()	3
Обработка команды links	4
main()	4
Клиентская часть (Swing GUI)	5
Класс Client	5
Подключение к серверу	5
Отправка сообщений	6
main()	6
Тестирование	6
Запуск сервера	6
Запуск клиента	6
Вывод	7

Цель работы

Разработать приложение для поиска ссылок в файлах с использованием ServerSocket и клиентского GUI-приложения на Swing. Сервер сканирует директорию, ищет ссылки формата [text](url) и возвращает их клиенту.

Задачи:

- Реализовать серверный FileExplorer для сканирования файлов.
- Обработать команду links для поиска ссылок.
- Создать клиентское GUI-приложение на Swing.

Серверная часть

Класс FileExplorer

Класс для работы с файловой системой:

server.cpp

```
class FileExplorer {
    const char *path;
    DIR *dir;

public:
    FileExplorer(const char *path) : path(path), dir(opendir(path))
    {}

    ~FileExplorer() { closedir(dir); }

    vector<string> ls() {
        struct dirent *entry;
        vector<string> result;
        while ((entry = readdir(dir)) != nullptr) {
            result.push_back(entry->d_name);
        }
        return result;
    }

    vector<string> contains(const vector<string> &list, const string
&s) {
        vector<string> result;
        for (auto str : list)
            if (str.find(s) != string::npos)
                result.push_back(str);
        return result;
    }

    vector<string> cat(const vector<string> &list) {
        vector<string> result;
        for (auto filename : list) {
            filename = string(path) + "/" + filename;
            std::ifstream file(filename);
            std::string line;
            while (std::getline(file, line))
                result.push_back(line);
            file.close();
        }
        return result;
    }
};
```

Методы:

- ls() — список файлов в директории
- contains() — фильтрация по расширению
- cat() — чтение содержимого файлов

Функция parse()

Поиск ссылок формата [text](url):

server.cpp

```
void parse(const string &s, int *x, int *y) {
    for (size_t i = 0; i < s.size(); i++) {
        if (s.at(i) == '[') {
            for (size_t j = i; j < s.size(); j++) {
                if (s.at(j) == ']') {
                    if (j + 1 < s.size() && s.at(j + 1) == '(') {
                        for (size_t k = j; k < s.size(); k++) {
                            if (s.at(k) == ')') {
                                *x = i;
                                *y = k - i + 1;
                                return;
                            }
                        }
                    }
                }
            }
        }
    }
    *x = -1;
    *y = -1;
}
```

Поиск осуществляется посимвольно:

- ищем [
- затем
-]
- затем (
- затем)
- возвращаем позицию и длину ссылки

Обработка команды links

server.cpp

```
size_t cmd_pos = str.find("links");
if (cmd_pos != std::string::npos) {
    FileExplorer fe("src");

    vector<string> links;

    for (const auto &s : fe.cat(fe.contains(fe.ls(), ".md"))) {
        int i, j;
        parse(s, &i, &j);
        if (i != -1)
            links.push_back(s.substr(i, j));
    }

    std::string result;
    for (const auto &s : links) {
        result += s + "\n";
    }

    send(clientSocket, result.c_str(), result.size(), 0);
}
```

Сервер:

1. Принимает команду «links»
2. Сканирует директорию src
3. Читает все .md файлы
4. Ищет ссылки формата [text](url)
5. Возвращает список ссылок

main()

server.cpp

```
int main() {
    Socket s(3896, "0.0.0.0");
    s.bind_and_listen();
    std::cout << "[*] Server started ..." << std::endl;
    while (true) {
        s.wait_for_message();
    }
}
```

Клиентская часть (Swing GUI)

Класс Client

Client.java

```
public class Client extends JFrame {  
    private JTextField serverInput;  
    private JTextField portInput;  
    private JButton connectButton;  
    private JTextArea responseArea;  
    private JTextField messageInput;  
    private JButton sendButton;  
  
    private Socket socket;  
    private PrintWriter out;  
    private BufferedReader in;  
}
```

Компоненты GUI:

- serverInput — адрес сервера
- portInput — порт сервера
- connectButton — кнопка подключения
- responseArea — область вывода ответов
- messageInput — поле ввода команды
- sendButton — кнопка отправки

Подключение к серверу

Client.java

```
private void connectToServer() {  
    String server = serverInput.getText().trim();  
    int port = Integer.parseInt(portInput.getText().trim());  
  
    socket = new Socket(server, port);  
    out = new PrintWriter(socket.getOutputStream(), true);  
    in = new BufferedReader(new  
InputStreamReader(socket.getInputStream()));  
  
    connectButton.setText("Подключено");  
    connectButton.setEnabled(false);  
    responseArea.append("Подключено к " + server + ":" + port +  
"\n");  
}
```

Отправка сообщений

Client.java

```
private void sendMessage() {
    String message = messageInput.getText();
    out.println(message);
    char[] buffer = new char[1024];
    int bytesRead = in.read(buffer);
    if (bytesRead > 0) {
        String response = new String(buffer, 0, bytesRead);
        responseArea.append("Сервер: " + response + "\n");
    }
    messageInput.setText("");
}
```

main()

Client.java

```
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        try {
            UIManager.setLookAndFeel(new FlatDarkLaf());
        } catch (Exception ignored) {}
        new Client().setVisible(true);
    });
}
```

Тестирование

Запуск сервера

server@host

```
$ make
$ ./server
[*] Server started ...
```

Запуск клиента

Пользователь вводит команду «links» и получает список ссылок из .md файлов.

client

```
links  
[text](https://example.com)  
[документация](doc.md)
```

Вывод

В ходе лабораторной работы были реализованы:

1. Сервер FileExplorer — сканирование директорий и поиск ссылок в файлах
2. Функция `parse()` — поиск ссылок формата Markdown `[text](url)`
3. Клиентский GUI на Swing — удобный интерфейс для отправки команд
4. FlatLaF — современное оформление интерфейса

Сервер обрабатывает команду «links», сканирует .md файлы и возвращает найденные ссылки.